

Руководство по выживанию — шифрование, аутентификация

 pro-ldap.ru/tr/zytrax/tech/encryption.html

Это руководство по выживанию охватывает такие ошеломительные темы как криптография, шифрование, авторизация и аутентификация. Для тех, кто сомневается в своей математической подготовке (и даже для тех, кто не сомневается) хочется отметить, что используемая в криптографии математика может ужаснуть кого угодно, и в этом руководстве она либо вообще не рассматривается, либо рассматривается поверхностно. Данное руководство сконцентрировано на функциональности системы и состоит исключительно из описаний, иллюстраций и разъяснений, а не из конкретных команд или реализаций. Большинство из рассматриваемых здесь вопросов служат основой для материалов статьи ["TLS/SSL и сертификаты X.509"](#), где хватает таких описаний реализаций и команд, которые способны надолго вывести кого угодно из состояния душевного равновесия.

Содержание:

- [Терминология](#)
- [Общий обзор криптографии](#)
- [Симметричное шифрование](#)
- [Асимметричное шифрование](#)
- [Обмен ключами по алгоритму Диффи-Хеллмана \(и его вариант с эфемерными ключами\)](#)
- [Отпечатки и хэши](#)
- [Аутентификационный код сообщения \(MAC\)](#)
- [Цифровые подписи](#)

Терминология

Неполный перечень терминов, используемых в защите информации. Многие из этих терминов разъясняются далее в тексте, другие остались сами по себе и приведены здесь отчасти для того, чтобы напугать читателя, но, главным образом потому, что мы понятия не имеем, что они означают.

Аутентификация (Authentication)	Процесс или процедуры, используемые для проверки того, что данные или информация, заявленные как поступившие из какого-то источника, могли поступить только из этого источника. Методы, используемые для аутентификации, включают цифровые подписи, MAC и даже добрые старые пароли. Как правило, после аутентификации Вам ещё нужно быть авторизованным для доступа к ресурсам или информации.
Авторизация (Authorization)	После того, как пользователи прошли аутентификацию, они, как правило, авторизованы (на основании их учётных данных для входа или свойств учетной записи) на получение или запрет доступа к определённым системным ресурсам, таким как файлы, приложения или сетевые ресурсы. Термин привилегия иногда используется как синоним авторизации. Так, пользователь может иметь достаточно привилегий для доступа к ресурсу X, но не к ресурсу Y, или, иными словами, он авторизован на доступ к X, но не авторизован на доступ к Y.
Асимметричная криптография (Asymmetric)	Раскрывается далее по тексту.
Битовая сложность, битовая стойкость (Bit Strength)	Определяется в битах и является мерой объема работ, которые необходимо выполнить для взлома криптографического алгоритма. Битовая сложность предоставляет метод, с помощью которого можно измерить относительную стойкость одного алгоритма шифрования к другому. В зависимости от алгоритма, размер ключа и битовая сложность могут совпадать или не совпадать. В разделе 5.6.1 части первой публикации NIST SP 800-57 битовая сложность определяется в упомянутых подробностях. Согласно текущих рекомендаций NIST (на 2010-2030 годы), минимальная битовая сложность алгоритма шифрования должна составлять не менее 112 бит. Далее на этой странице приведена упрощенная таблица сопоставления битовой сложности и размеров ключей.
Шифр, шифрование (Cipher)	Шифр иначе называется алгоритмом шифрования. В английском языке под этим термином также иногда понимается процесс шифрования.

Открытый текст (Clear Text или Plain Text)	Блок данных, к которому не применялся процесс шифрования, либо блок данных, полученный в результате операции расшифрования или дешифрования.
Удостоверяющие данные (Credentials)	Причудливое название того, что большинство из нас назвало бы паролем (хотя они могут принимать и иные формы, такие как аппаратный токен или биометрические данные). Ваши удостоверяющие данные являются одним из способов доказать, что Вы именно тот, за кого себя выдаёте. Так как Вы должны быть единственным человеком (или, в некоторых случаях, группой лиц) кто знает или имеет доступ к Вашим удостоверяющим данным, то когда Вы предоставляете их в системе или в сети это доказывает, что Вы именно тот, за кого себя выдаёте. После выполнения неких форм обмена данными, которые будут включать в себя предоставление Ваших удостоверяющих данных (например, набор пароля) Вы становитесь аутентифицированы . Как правило, после аутентификации Вам ещё нужно быть авторизованным для доступа к ресурсам или информации.
Расшифрование (Decipher)	Применение алгоритма расшифрования к зашифрованному блоку или тексту с предоставлением соответствующих ключей, в результате чего он превращается в открытый текст. Многие (но не все) шифры используют один и тот же алгоритм для шифрования и расшифрования. <i>Примечание переводчика: в русскоязычных криптографических кругах принято под термином "расшифрование" понимать процесс получения открытого текста с использованием ключа и алгоритма расшифрования, предусмотренного для данного шифра, а под термином "дешифрование" (или "дешифровка") — процесс получения открытого текста средствами криптоанализа (взлома). Смотрите аргументы "за" и "против" подобного подхода тут. Поскольку данный перевод рассчитан на русскоязычную аудиторию, в нём соблюдается "отечественная" практика применения этого термина.</i>
Дайджест (Digest)	Смотрите описание термина хэш .
Шифрование (Encryption)	Процесс преобразования данных с использованием шифра (или алгоритма шифрования). Используемые методы могут быть симметричными или асимметричными .

Отпечаток (Fingerprint)	Смотрите описание термина хэш .
Прямая секретность (Forward Secret или Forward secrecy)	Это понятие относится к характеристике, при которой злоумышленник, скомпрометировав долговременный секретный ключ (например, закрытый ключ алгоритма асимметричного шифрования), не сможет дешифровать предыдущие переговоры или сессии. Обычно (например, в TLS/SSL) это делается путём вычисления сеансовых ключей с применением случайного значения.
Хэш (Hash)	Он же — дайджест (digest), дайджест сообщения (message digest), односторонний хэш (one-way hash) или отпечаток (fingerprint). Алгоритм для уменьшения бесконечно большого блока данных до уникальной строки октетов значительно меньшего и фиксированного размера, например, 128 или 256 бит (16 или 32 байт). Иногда используется термин <i>односторонний хэш</i> (one-way hash), поскольку он чётко указывает на то, что исходное сообщение не может быть воссоздано из хешированной формы. Хэши используются для обеспечения целостности сообщений, а также в MAC и цифровых подписях . Термин <i>отпечаток</i> (fingerprint, дословно <i>отпечаток пальца</i>) иногда используется в том смысле, что отпечаток пальца в определённых случаях может заменить всего человека, то есть хэш становится тем отпечатком, который может храниться и передаваться независимо от сообщения или данных, но всегда может быть использован для проверки целостности этого сообщения или данных.
Размер ключа (Key Size)	Иначе называется длиной ключа. Количество бит, используемых для ключа алгоритма шифрования. В общем случае, чем больше бит, тем более стойким является алгоритм. Многие алгоритмы могут принимать ключи переменного размера, что позволяет им время от времени постепенно увеличивать свою стойкость. Длину (размер) ключа не следует путать с битовой сложностью криптографической реализации. Так, в RSA длина (размер) ключа — 2048 бит, а битовая сложность — 112 бит.
Дайджест сообщения (Message Digest)	Смотрите описание термина хэш .

Неотказуемость (Non-Repudiation)	Если некоторый объект, — человек или система, — прошёл проверку подлинности (аутентификацию) как это описано выше, то считается доказанным, что он является тем, за кого себя выдаёт. Он не может впоследствии отрицать (не признавать) факт прохождения данной аутентификации. Таким образом, если объект успешно аутентифицируется, скажем, в финансовой системе и снимает 10 миллиардов чатлов, он не может впоследствии отрицать (отвергать) эту транзакцию.
Односторонний хэш (One-way Hash)	Смотрите описание термина хэш .
Пароль (Password)	Смотрите описание термина удостоверяющие данные .
Общий секрет (Shared Secret)	Имеется ввиду единственный ключ, применяемый в алгоритмах симметричного шифрования , когда все участники секретных переговоров должны иметь копию одного и того же ключа (совместно его использовать).
Симметричная криптография (Symmetric)	Раскрывается далее по тексту .

Общий обзор криптографии

Согласно словарю Уэбстера криптография — это "шифрование и расшифрование сообщений с помощью секретного кода или шифра; также: компьютеризированное кодирование и декодирование информации".

Криптографией называют процесс преобразования (шифрования) данных, представленных "открытым текстом", с помощью некоторого метода (шифра или алгоритма шифрования) в абракадабру, которую можно трансформировать (расшифровать) обратно в открытый текст, только если получатель обладает неким секретным знанием, таким как ключ или набор ключей.

Исторически секретное знание представляли из себя сами шифры или алгоритмы шифрования. Например, такой шифр, как замена каждой буквы на предшествующую ей в алфавите, который порой используют дети или мы сами, когда посылаем секретные послания своим друзьям. Слабое место такого подхода в том, что если метод или алгоритм шифрования раскрыт, все коммуникации, зашифрованные с помощью этого алгоритма или шифра, могут быть преобразованы в обычный текст

(дешифрованы). Нужно будет разработать новый алгоритм и распространить по всем заинтересованным сторонам перед тем, как мы можем снова начать посылать сообщения.

Современная криптография предполагает, что злоумышленники рано или поздно раскроют криптографический алгоритм. По причинам, которые мы увидим позднее, в действительности сами алгоритмы находятся в свободном доступе. Секретное знание же теперь состоит в уникальном ключе или ключах, используемых этими алгоритмами для преобразования (шифрования и расшифрования) данных. Если этот ключ скомпрометирован (проще говоря, украден злоумышленником), то путём простой замены ключа, оставляя алгоритм без изменений, можно возобновить конфиденциальные коммуникации. Злоумышленнику ничего не остаётся, как приниматься за раскрытие ключа заново, причём знаний у него будет не больше, чем перед началом предыдущего взлома, а мы, в свою очередь, постараемся ужесточить процедуры обеспечения безопасности ключевой информации.

Криптографические алгоритмы (в математическом смысле) не являются абсолютно закрытыми, наоборот, информация о них обычно находится в свободном доступе и они вновь и вновь подвергаются атакам определённого круга исследователей и специалистов (тестеров-взломщиков), которые жить без этого не могут. Только те алгоритмы, которые длительное время выдерживали повторяющиеся атаки, используются затем в промышленной эксплуатации. Поскольку исследования в сфере криптографических алгоритмов продолжаются, иногда это приводит к необходимости замены казалось бы зрелого и надёжного алгоритма, когда в нём обнаруживается уязвимость. В качестве недавнего примера здесь можно привести теоретические уязвимости, обнаруженные в алгоритме создания отпечатков MD5 в 2004 году.

Хотя нельзя исключать того, что для нахождения ключа злоумышленник может использовать атаки методом грубой силы (то есть простым перебором), криптографические системы используют концепцию, известную как **вычислительная невозможность** (термин, придуманный Диффи и Хеллманом в их оригинальной статье), которая, попросту, означает, что атака методом грубой силы будет потреблять неприемлемо большое количество ресурсов и времени. Вычислительная невозможность основывается на достигнутом в определённый момент уровне технологий и, следовательно, не является абсолютной величиной. Так, например, в некоторых алгоритмах время от времени, по мере увеличения вычислительных мощностей, увеличивается и размер ключа.

Если злоумышленники получили секретный ключ или ключи (путём воровства, подбора, удачи или иных мелкопакостных средств), они конечно же не станут трубить об этом на весь мир, чтобы пользователь сразу же поменял ключ (ключи). Вместо этого они продолжают тихо и спокойно прослушивать якобы безопасные

коммуникации. Это, конечно же, является серьёзной проблемой и обычно решается некоей комбинацией содержания ключей в среде "доказанного вмешательства (tamper-proof)" (которая уничтожит ключи при попытке компрометации) или "оповещения о вмешательстве (tamper-aware)" (она же HSM, которая предупредит пользователя, если ключ скомпрометирован), либо путём регулярной смены ключей, — так называемым обслуживанием ключей, — которая просто минимизирует потенциальные риски. До сих пор единственным способом **точно узнать** или **доказать** факт компрометации ключа является наблюдение за эффектом (обычно негативным), возникающим вследствие такой компрометации.

Замечание по эксплуатации криптосистем: Многие криптографические алгоритмы существуют и остаются неизменными на протяжении многих лет (в некоторых случаях больше десятилетия). Было написано много стандартов, предлагающих ряд криптографических алгоритмов, но, как правило, обязательным из них являлся только один, а остальные нужны лишь для поддержания некоторой формы "общего знаменателя". Однако, поскольку скорости вычислений возрастают, а криптоатаки учащаются (иногда от источников, никогда не вызывавших опасений), всё большее значение приобретает необходимость менять либо алгоритмы, либо размеры ключей. Этот процесс, на жаргоне называемый *алгоритмической гибкостью*, может стать серьёзной проблемой в эксплуатации систем, в которых используются "старые" версии алгоритмов или ключей.

Криптография может использоваться для решения трёх задач:

1. **Конфиденциальность:** Только стороны, участвующие в коммуникации, могут понять сообщения или данные, посылаемые между этими сторонами.
2. **Аутентификация:** Данные могут поступать только из распознанного источника.
3. **Целостность:** Данные, полученные одной стороной, являются именно теми данными, которые были отправлены другой стороной; во время передачи они не подвергались манипуляциям или компрометации.

Решение одной или нескольких из перечисленных задач может обеспечиваться применением как единственного алгоритма, так и комбинацией алгоритмов или методов.

Сначала обратимся к основным методам. Современные криптографические методы могут быть симметричными или асимметричными.

Примечание: Один из лучших источников информации по криптографии — [публикации Национального Института стандартов и технологий США \(NIST\)](#). В частности, в первой части SP800-57 (на текущий момент ревизия 4, 2016 год) обсуждается управление ключами и приводится превосходный и тщательный анализ

как методов криптографии, так и угроз. Кроме того, там даны (в таблицах 2 и 4) практические рекомендации по размерам ключей для различных алгоритмов. Заинтересованным читателям мы настоятельно рекомендуем хорошенько изучить этот достойный, хотя и большой, документ, как основательное и весьма практичное посвящение в тайные знания криптографии. На этой странице мы ссылаемся также на ряд других документов NIST SP.

Наконец, нет ничего лучше для любознательного читателя, чем прочесть научную статью Уитфилда Диффи и Мартина Хеллмана **"Новые направления в криптографии"** (1976 г.), с которой началась революция применения систем с открытым ключём. Математическая часть в ней несколько тяжеловата, но, к счастью, её можно пропустить, не теряя при этом кристальной ясности идей. Прекрасный образец хорошо читаемой прозы, что выгодно выделяет её среди такого рода статей. Проведёте время с большой пользой.

Симметричная криптография

Симметричные алгоритмы шифрования, называемые также системами с одним ключом (single-key), с общим секретом (shared-secret), или даже системами с закрытым ключом (private-key), используют единственный ключ (или набор ключей) для шифрования и расшифрования данных. Единственный ключ (общий секрет) должен быть безопасным образом распространён между сторонами, которые будут его использовать, перед началом самих безопасных коммуникаций. У систем с общим секретом существует как минимум два узких места. Во-первых, ключ должен быть распространён безопасным способом с помощью процесса, называемого управлением ключами, который сам по себе нетривиален. Во-вторых, ответственность за безопасность уже распространённого ключа лежит на всех сторонах коммуникации: "Самому себе я доверяю, но могу ли я доверять всем остальным сторонам, что они не допустили огласки ключа?". Если ключ, являющийся общим секретом, скомпрометирован на любой из сторон, то он скомпрометирован для всех использующих его сторон. Симметричные алгоритмы используют значительно меньше вычислительных ресурсов, нежели их асимметричные коллеги. Как правило, они являются единственным приемлемым способом шифрования объемных потоков данных.

Примеры распространённых симметричных алгоритмов шифрования: DES (Data Encryption Standard, он же Data Encryption Algorithm (DEA)), Triple DES (TDES, он же TDEA (Triple DEA)), AES (Advanced Encryption Standard), IDEA (International Data Encryption Algorithm), а также RC4 (Rivest Cipher 4 — по состоянию на 2013 год рассматривается как потенциально подверженный взлому, хотя факты успешных атак ещё не доказаны и не опубликованы), типичные размеры ключей 64, 128, 192 или 256 бит. На рисунке 1 демонстрируется рабочее применение общего секрета в

классических конфиденциальных коммуникациях. **Примечание:** Термин "общий секрет (shared secret)", описывающий единственный ключ (или набор ключей), используемый или разделяемый обеими сторонами коммуникации, не следует путать с "разделением секрета (secret sharing)", описывающим процесс, при котором общий или единственный секретный ключ разбивается на части и распределяется между несколькими лицами, чтобы сделать его более защищённым.

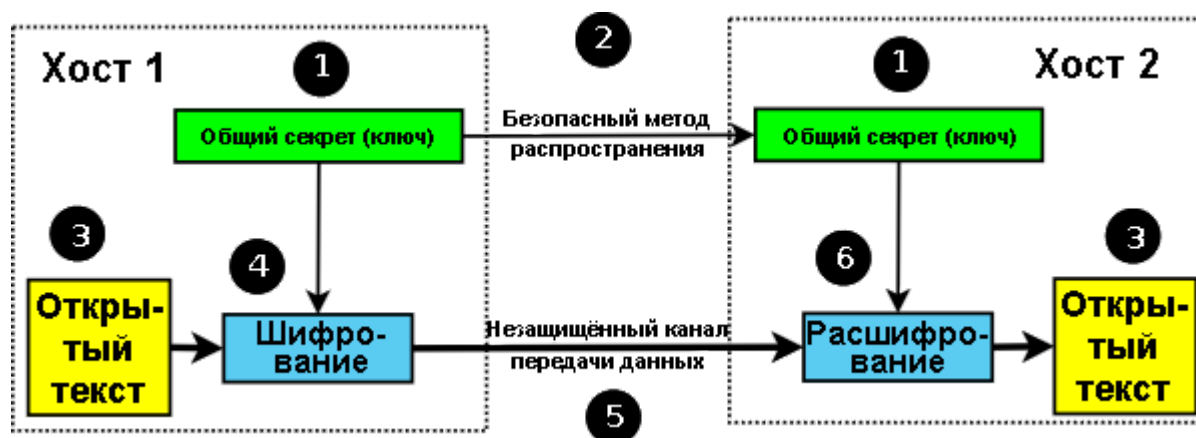


Рисунок 1 — Симметричная криптография

Асимметричная криптография

Асимметричные алгоритмы шифрования используют пару ключей, — открытый и закрытый, — и обычно называются криптографическими системами с открытым ключом или, иногда, несекретным шифрованием (своего рода оксюморон). В таких системах данные (на жаргоне — открытый текст), зашифрованные одним ключом, могут быть расшифрованы только парным ему ключом. Имея один ключ, вычислительно невозможно вывести парный ему ключ. Принцип работы асимметричной криптографии заключается в предоставлении одного ключа, называемого открытым, в широкий доступ, при сохранении другого ключа, называемого (сюрприз!) закрытым, в тайне. У этой технологии есть интересный побочный эффект. Если сообщение было зашифровано закрытым ключом и может быть расшифровано парным ему открытым ключом, то только владелец закрытого ключа мог произвести шифрование. Это свойство используется в [цифровых подписях](#) и описано ниже. По сравнению со своими симметричными коллегами, асимметричные алгоритмы потребляют значительное количество вычислительных ресурсов и потому обычно не применяются для шифрования объёмных потоков данных.

Наиболее широкое распространение среди криптосистем с открытым ключом получили RSA (по первым буквам фамилий разработчиков: Rivest, Shamir и Adelman), DSA (Digital Signature Algorithm, алгоритм цифровой подписи) и Криптография на основе эллиптических кривых (Elliptic Curve Cryptography, ECC или просто EC). Типичные размеры ключа в системах с открытым ключом RSA — 1024 или 2048 бит

(текущие рекомендации US NIST на период с 2010 по 2030 год), а то и больше, если кому-то доставляет удовольствие чрезмерно перегружать процессор при расшифровании. В ECC обычно используются чуть меньшие размеры ключа, например, 160, 233, 283, 384 или 571 бит. Открытый ключ из пары закрытого и открытого ключей может без нарушений безопасности храниться в общедоступных сервисах, таких как DNS, а закрытый ключ должен храниться в надёжном защищённом месте (возможно даже в аппаратных модулях безопасности (Hardware Security Modules, HSM)). На рисунке 2 демонстрируется использование криптографии с открытым ключом для классических конфиденциальных коммуникаций.

Примечание: Хотя в текущих рекомендациях NIST (до 2030 года) указана длина ключа RSA 2048 бит, в [RFC 8603](#) выдвигается обязательное требование использовать ключи RSA длиной 3072 или 4096 бит (или ECDSA P-384) для сертификатов X.509, используемых в рамках CNSA Suite (Commercial National Security Algorithm Suite).

[Таблица сопоставления битовой сложности и размеров ключей.](#)

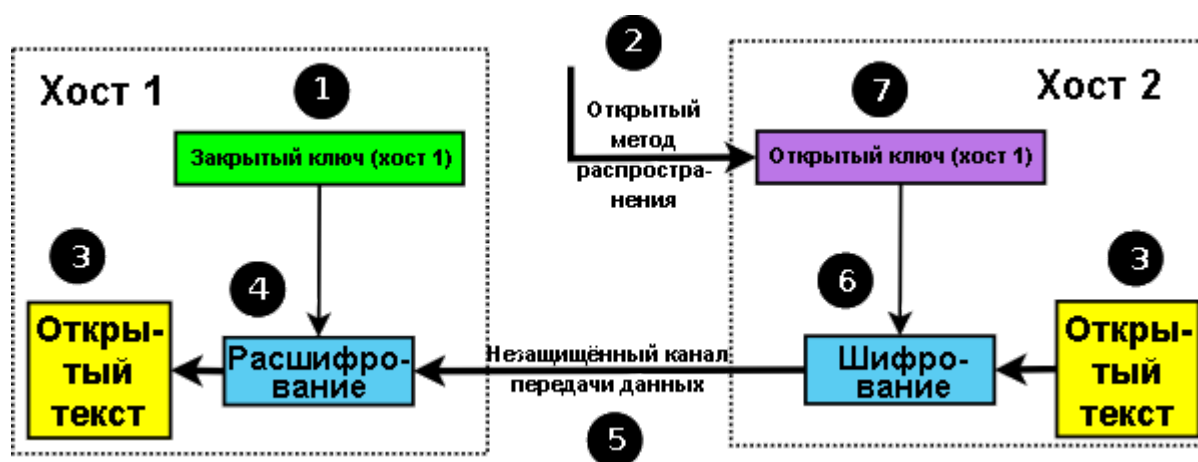


Рисунок 2 — Асимметричная криптография

Для достижения конфиденциальности сообщение, посылаемое от хоста 2 на хост 1 зашифровывается с использованием открытого ключа хоста 1. Это сообщение можно расшифровать только с помощью закрытого ключа хоста 1. Если хосту 1 требуется отправить конфиденциальное сообщение на хост 2, то он должен получить открытый ключ хоста 2 (не показано на рисунке 2, чтобы не загромождать).

У систем с открытым ключом есть одно существенное ограничение. Они полагаются на знание или доверие тому, что открытый ключ, который будет использоваться в коммуникациях с человеком, организацией или объектом, действительно является открытым ключом этого человека (организации или объекта) и не был подменён злонамеренной третьей стороной. Обычно это достигается двумя основными методами: инфраструктурой открытых ключей (Public Key Infrastructure, PKI), или, в более общем случае, использованием доверенной третьей стороны. Третья сторона

безопасно управляет открытыми ключами и подтверждает их подлинность. Если некто запрашивает у третьей стороны (в контексте сертификатов X.509 называемой удостоверяющим центром (Certificate Authority, CA)) открытый ключ объекта X, он доверяет, что ему был предоставлен корректный открытый ключ. Третья сторона, на основании проведенных проверок подлинности (аттестаций, нотариальных заверений и других), достоверно заявляет, что X — это единственный глобально-уникальный объект X. Наиболее распространённый способ информирования о том, что открытые ключи были удостоверены третьей стороной — встраивание их в [сертификаты X.509 \(или SSL\)](#), подписанные цифровой подписью издателя сертификата (обычно удостоверяющего центра).

Замечания относительно производительности RSA и криптографии на основе эллиптических кривых

RSA существует примерно с 1977 года, а криптография на основе эллиптических кривых (ECC) — с 1985. В связи с широким распространением RSA (в 2018 году это был доминирующий метод шифрования с открытым ключом) он был широко исследован на предмет уязвимостей, и для большинства архитектур машин существуют оптимизированные реализации. ECC исследован гораздо меньше и, следовательно, всё ещё может преподнести какие-нибудь сюрпризы (а может и не преподнести). Реализации ECC, по всей видимости, ещё не достигли максимальной эффективности, и потому к приведённым ниже сравнениям следует относиться с некоторой осторожностью, но, скорее всего, в целом они верны.

Функционал	Сравнение
Размер ключа	При одной и той же битовой сложности размер ключа в ECC примерно на 70% меньше, чем в RSA. В этом отношении преимущество ECC возрастает с увеличением битовой сложности .
Генерация ключа	При одной и той же битовой сложности в ECC генерация выполняется примерно в 10 раз быстрее, чем в RSA. В этом отношении преимущество ECC возрастает с увеличением битовой сложности .
Генерация подписи (шифрование)	RSA на несколько порядков быстрее, чем ECC, независимо от битовой сложности .
Проверка подписи (расшифрование)	Вплоть до битовой сложности в 112 бит скорость RSA примерно совпадает с ECC, а дальше ECC становится быстрее, и его преимущество возрастает с увеличением битовой сложности .

Скорость в целом (шифрование/ расшифрование)	Вплоть до битовой сложности в 128 бит скорость RSA примерно совпадает с ECC, а дальше ECC становится быстрее, и его преимущество возрастает с увеличением битовой сложности .
--	---

Не надо быть семи пядей во лбу, чтобы догадаться, что мобильные устройства будут предпочитать ECC из-за меньшей нагрузки на процессор и меньших требований к пропускной способности при приёме, скажем, веб-страницы (когда осуществляется расшифрование), а веб-серверы столкнутся со значительным увеличением нагрузки из-за повышенных требований шифрования ECC к производительности, и, вероятно, будут стремиться продолжать использовать RSA.

Сопоставление битовой сложности и размеров ключей

Таблица взята из [части 1 публикации NIST SP 800-57](#), раздел 5.6.1, таблица 2.

Битовая сложность	RSA	ECC
80	1024	160-233
112	2048	224-255
128	3072	256-383
7680	7680	384-511

Асимметричные алгоритмы и квантовые компьютеры

В асимметричных алгоритмах (как в RSA, так и в ECC) используется сложная целочисленная математика (как правило, на основе простых чисел) и потому в мире квантовых вычислений они становятся весьма уязвимыми и небезопасными. Постепенно появляется новое поколение алгоритмов, которые будут работать в квантовом мире. В [RFC 8554](#) описан алгоритм, известный как LMS (Leighton-Micali Signature), в котором используются одноразовые подписи (One-Time Signature, OTS). Есть надежда, что он выживет в квантовом мире. В связи с отсутствием опыта эксплуатации подобных систем, от дизайнеров и разработчиков требуется тщательное рассмотрение возможных последствий использования этого нового поколения алгоритмов, и потому такие RFC публикуются в экспериментальных целях и имеют **информационный** статус. Однако, их значимость не следует недооценивать.

Обмен ключами по алгоритму Диффи-Хеллмана (и его вариант с эфемерными ключами)

Алгоритм обмена Диффи-Хеллмана (DH) — это метод, по которому две (а в некоторых случаях и более) сторон могут независимо создать один и тот же общий секрет, который затем может использоваться в [симметричных](#) криптографических алгоритмах (в TLS, например, DH используется для создания ключа, применяемого в фазе протокола записи данных). Предполагается, что весь сеанс, в течение которого происходит коммуникация двух сторон, может быть перехвачен третьей стороной. Эта третья сторона не сможет вывести тот же самый ключ, поскольку для этого ей не будет хватать некоторой информации.

В оригинальном алгоритме обмена Диффи-Хеллмана (сейчас его часто называют *протоколом Диффи-Хеллмана в конечном поле*) используется экзотическая математика:

Y (общий секрет) = g (генератор) $^ X$ (секретный ключ) $\bmod p$ (простое число — модулятор)

Если затем мы определим X как **(закрытый ключ стороны 1) $^$ (закрытый ключ стороны 2)** (путём математических выкладок можно показать, что его также можно записать как **(закрытый ключ стороны 2) $^$ (закрытый ключ стороны 1)**), тогда мы сможем организовать процедуру открытого обмена, которая не позволит злоумышленнику, занимающемуся прослушиванием трафика, найти общий секретный ключ.

Такой обмен продемонстрирован на рисунке 3.

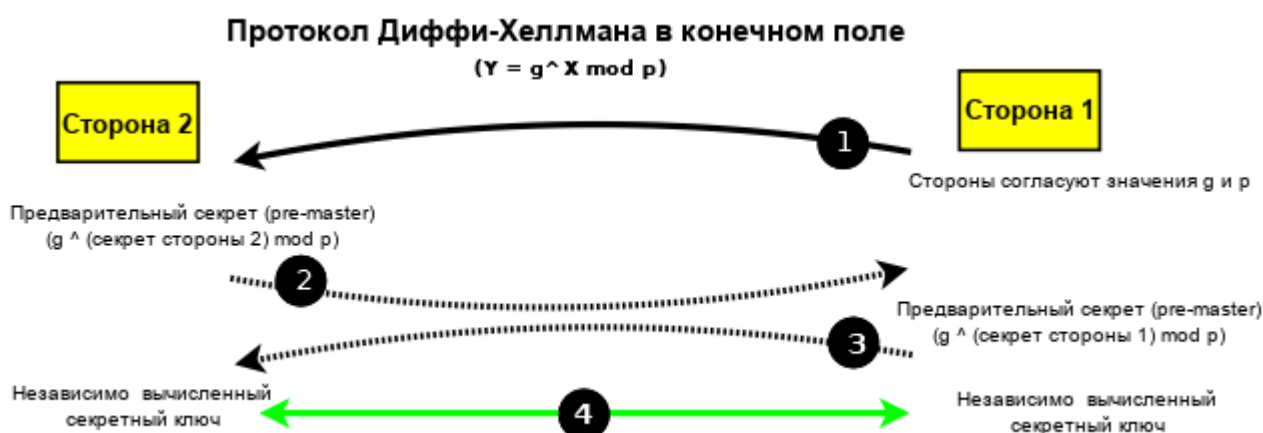


Рисунок 3 — Алгоритм обмена Диффи-Хеллмана

1. Две стороны обмена публично определяют значения p и g (1). В некоторых системах/протоколах инициатор обмена просто предоставляет p и g другой стороне, в других системах происходит некая форма взаимного согласования этих значений сторонами обмена.
2. Затем сторона 2 генерирует промежуточный результат (иногда называемый pre-master секретом) путём вычисления $g^a \pmod p$ (2), и может публично отправить его стороне 1. В некоторых использующих DH протоколах этот промежуточный результат перед отправкой может быть зашифрован.
3. В свою очередь сторона 1 вычисляет $g^b \pmod p$ (3), и может публично отправить его стороне 2 (опять же, в некоторых протоколах этот обмен может шифроваться). Порядок, в котором стороны 1 и 2 вычисляют промежуточные результаты и обмениваются ими, не существенен.

На шаге (4) сторона 1 берёт промежуточный результат стороны 2 и возводит его в степень своего (закрытого ключа стороны 1), то есть фактически вычисляет $g^{ab} \pmod p$ ($(g^b)^a \pmod p$) = Y (общий секрет). Независимо от этого сторона 2 берёт промежуточный результат стороны 1 и возводит его в степень своего (закрытого ключа стороны 2), то есть фактически вычисляет $g^{ab} \pmod p$ ($(g^a)^b \pmod p$) = то же значение Y (общий ключ). На рисунке 3 показано использование данного совместного (или общего) секрета для шифрования всего последующего трафика (4) с применением некоторого алгоритма [симметричного шифрования](#) (о котором стороны договорились ранее).

Если Вы интересуетесь тонкостями математики данного процесса (мы — не очень), то в этом [очень коротком видеоролике](#) она объясняется просто потрясающе.

Примечания:

1. В классическом протоколе Диффи-Хеллмана в конечном поле значения p и g обычно небольшие, а секретные ключи сторон 1 и 2, наоборот, большие, порой просто огромные.

2. Если обе стороны обмена каждый раз используют один и тот же секретный ключ, тогда при компрометации любого из этих ключей текущий обмен, а также все предыдущие обмены могут быть дешифрованы. На криптографическом жаргоне можно сказать, что не обеспечивается прямая секретность. Чтобы этого избежать, одна или обе стороны обмена могут генерировать новый закрытый ключ при каждом обмене по алгоритму Диффи-Хеллмана. Это называется *Эфемерным обменом по алгоритму Диффи-Хеллмана (Ephemeral Diffie-Hellman exchange)*, часто для его обозначения используется сокращение DHE, не совсем, на наш взгляд, однозначное).
3. Всё чаще используются алгоритмы, отличные от DH в конечном поле, в частности, эллиптические кривые (EC). Во время первоначального согласования (1) стороны договариваются о типе кривой, а затем происходит публичный обмен конкретными параметрами кривой. Этот алгоритм получил название *Протокол Диффи-Хеллмана на эллиптических кривых (Elliptic curve Diffie-Hellman exchange)*, для его обозначения обычно используется сокращение (EC)DH, а при использовании эфемерного обмена — (EC)DHE.
4. Обмен по алгоритму Диффи-Хеллмана уязвим для атак типа "Человек посередине". Большинство протоколов, в которых применяется DH, используют какую-либо форму аутентификации. Например, в TLS обычно используется сертификат x.509 для аутентификации как минимум одной из сторон обмена.

Отпечатки сообщений (хэши)

Для обеспечения целостности данных сообщение можно просто зашифровать. Чтобы изменить содержащиеся в сообщении данные, злоумышленник должен обладать единственным ключом (в симметричных системах) или закрытым ключом (в асимметричных системах). Однако алгоритмы шифрования/расшифрования используют сложные математические функции и потому потребляют много ресурсов процессора. Шифрование всех сообщений подряд приводит к неприемлемо высокому потреблению ресурсов, и это особенно неприятно, когда конфиденциальность данных не требуется. К счастью, для уменьшения нагрузки можно использовать другие методы. Самый распространённый — нересурсоёмкая процедура, называемая односторонним хэшем, просто хэшем, или чаще **отпечатком сообщения (message digest)** (в обиходе просто **digest**). Алгоритм хэшей или отпечатков создаёт уникальный и относительно небольшого фиксированного размера (независимо от исходной длины сообщения) отпечаток (**digest**), вернуть который в исходное состояние невозможно. Полученные хэши или отпечатки иногда называются **отпечатками пальцев**, поскольку они уникально описывают исходный открытый текст. Отправляемое сообщение включает сразу и открытый (незашифрованный) текст и отпечаток этого сообщения. Алгоритм хэширования

применяется к полученному открытому тексту, и, если результат совпадает с полученным отпечатком сообщения, значит данные не были изменены. В некотором смысле отпечатки сообщений аналогичны по концепции с контрольными суммами, но значительно отличаются от них математическими свойствами.

Самые распространённые формы отпечатков сообщений — MD5 (Message Digest 5) и SHA-1, а в последнее время SHA-224, SHA-256, SHA-384 и SHA-512 (последние четыре — представители семейства алгоритмов SHA-2 (Secure Hash Algorithm 2)). В августе 2015 года NIST представил алгоритм SHA-3. Рисунок 4 демонстрирует использование отпечатков сообщений.

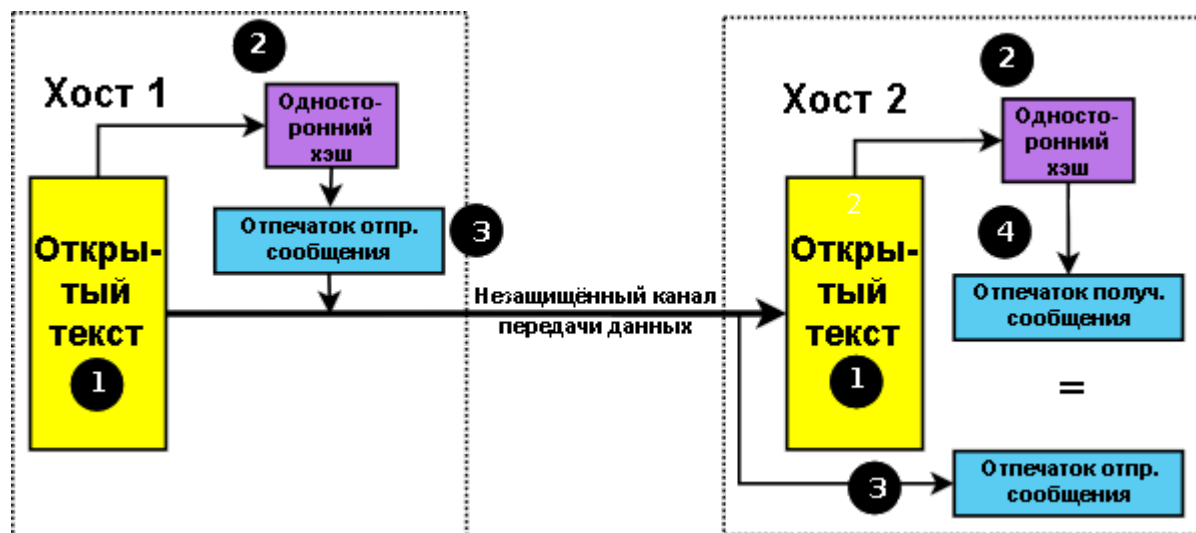


Рисунок 4 — Отпечатки сообщений

У простого добавления отпечатка сообщения к блоку открытого текста есть очевидное слабое место: путём замены сразу и открытого текста, и его отпечатка злоумышленник может подменить исходное сообщение или даже послать ложное сообщение от имени легального отправителя.

Примечание: Таблица 1 документа NIST SP800-107 содержит оценку относительной "силы" каждого алгоритма создания хэшей или отпечатков сообщений.

Аутентификационный код сообщения (Message Authentication Code, MAC)

Для аутентификации и поддержания целостности данных может использоваться так называемый аутентификационный код сообщения (MAC). MAC сочетает в себе отпечаток сообщения и общий (секретный) ключ. Часть, являющаяся ключом, аутентифицирует отправителя, а часть, представляющая собой хэш или отпечаток, обеспечивает целостность данных. Существуют две формы MAC. MAC, основанный на симметричном блочном шифре (таком как TDEA или AES), называется CMAC.

MAC, основанный на алгоритме хэширования (создания отпечатка сообщения), называется Hashed MAC (HMAC) и, вероятно, используется наиболее широко.

Самые распространённые формы HMAC — HMAC-MD5, HMAC-SHA-1, а в последнее время HMAC-SHA-224, HMAC-SHA-256 и HMAC-SHA-384. На рисунке 5 показано, как используется MAC. Применяемый в HMAC **закрытый ключ** может быть сгенерирован различными методами (определёнными в SP 800-107, ревизия 1). На упрощённом уровне этот ключ HMAC может быть рассмотрен как простая **"соль"**, назначение которой — аутентифицировать источник. **Примечание:** Алгоритм хеширования MD5 и, косвенно, любые использующие его алгоритмы, такие как HMAC-MD5, получили в большинстве документов IETF статус "нерекомендуемые" в связи с некоторыми теоретическими уязвимостями, опубликованными в начале 2005 года. Тем не менее, наличие этих недостатков не говорит о том, что использовать данные алгоритмы нельзя. Скорее это значит, что, как только появится возможность, следует перейти на более безопасные HMAC (семейство HMAC-SHA-2). К сожалению, во многих старых системах единственным поддерживаемым алгоритмом HMAC может быть HMAC-MD5.

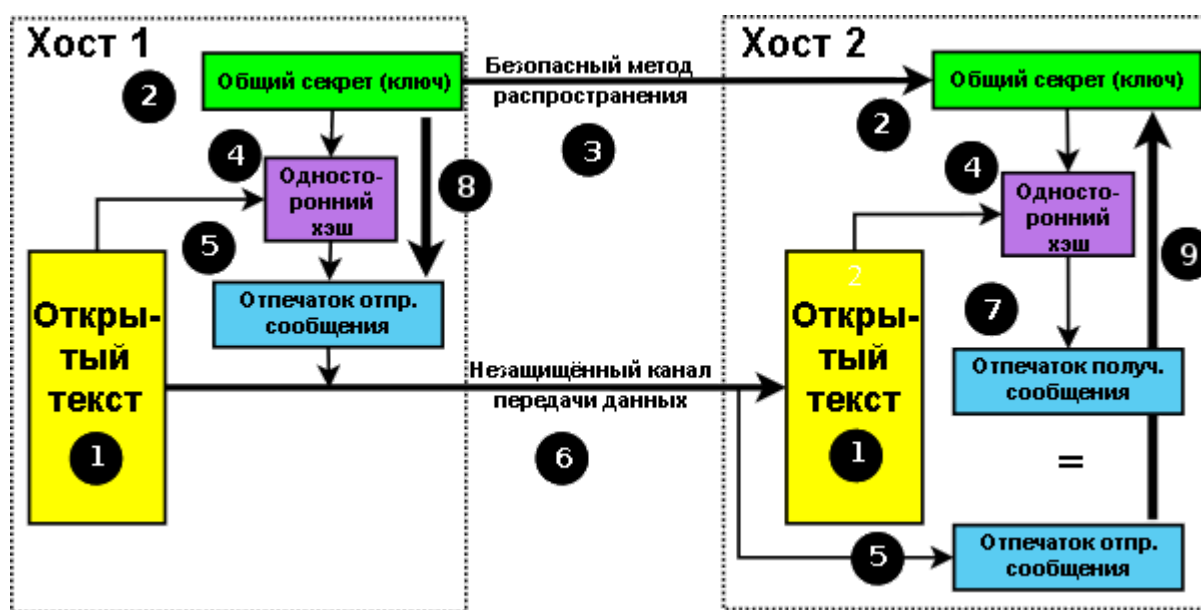


Рисунок 5 — Аутентификационный код сообщения (MAC)

Секретный ключ (2) распространяется с помощью некой процедуры распространения ключей (3) до начала взаимодействия. Открытый текст (1) обрабатывается алгоритмом хеширования (4) для создания отпечатка отправляемого сообщения (5), который затем шифруется (8) с использованием выбранного алгоритма и ключа (2). Затем сообщение и его зашифрованный отпечаток отправляется получателю по незашифрованному каналу передачи (6). Получатель берёт открытый текст (1) и пропускает его через тот же алгоритм хэширования (4) для создания отпечатка полученного сообщения (7). Принятый зашифрованный отпечаток отправляемого сообщения (5) расшифровывается (9) с использованием выбранного алгоритма и

ключа (2), и полученный в результате отпечаток отправляемого сообщения (в открытом виде) сравнивается с отпечатком полученного сообщения (7). Совпало — все в шоколаде, не совпало — сами знаете в чём.

На рисунке 5 показано шифрование только отпечатка или хэша. В результате мы получаем гарантию целостности сообщения. Если требуется также обеспечение конфиденциальности сообщения, то можно зашифровать его целиком (сразу и открытый текст, и хэш, или отпечаток). Протокол [TLS \(SSL\)](#) обеспечивает эту (последнюю) возможность в фазе записи данных.

<заморочки с криптографической терминологией> Показанный на рисунке 5 ключ назван **секретным ключом**, но это также и **закрытый ключ**. Он назван **секретным ключом**, поскольку для того, чтобы оставаться в тайне, он должен постоянно быть закрытым. Аналогично, и даже более корректно, он может быть назван **закрытым ключом** (хотя термин **закрытый ключ** имеет подтекст асимметричной криптографии), поскольку он должен постоянно быть закрытым для того, чтобы оставаться в тайне. Иногда в жизни так трудно разобраться...**</заморочки с криптографической терминологией>**

Цифровые подписи

В мире асимметричной криптографии, или криптографии с открытым ключом, для аутентификации и поддержания целостности данных используются так называемые цифровые подписи. Отправляемое сообщение, опять же, хэшируется с помощью, скажем, MD5, SHA-1, SHA-256 или SHA-384, чтобы можно было убедиться в целостности данных. Полученный в результате отпечаток сообщения затем зашифровывается с помощью закрытого ключа отправителя (с точностью до наоборот относительно обеспечения конфиденциальности в криптографии с открытым ключом, смотрите примечание ниже). Получателю отправляются и сообщение в виде открытого текста, и зашифрованный отпечаток. Получатель расшифровывает отпечаток сообщения с помощью открытого ключа отправителя, применяет алгоритм хэширования к полученному открытому тексту, и, если результаты совпали, то подтверждается сразу и подлинность отправителя, и целостность данных.

Наиболее распространённые алгоритмы цифровых подписей — RSA-MD5, RSA-SHA-1, RSA-SHA-256, RSA-SHA-384, DSA (Digital Signature Architecture, стандарт правительства США, определён в FIPS-186, ревизия 4) и ECDSA (Elliptic Curve Digital Signature Algorithm, определён в FIPS-186, ревизия 4). Типичные размеры ключа для систем цифровых подписей RSA и DSA — 768, 1024, а в последнее время 2048 бит (текущие рекомендации US NIST на период с 2010 по 2030 год), а то и больше. Размеры ключа цифровых подписей ECDSA для аналогичного уровня обеспечения безопасности значительно меньше и обычно составляют 112, 128, 192 или 256 бит.

На рисунке 6 показано, как используется цифровая подпись. **Примечание:** Алгоритм хеширования MD5 и, косвенно, любые использующие его алгоритмы, такие как RSA-MD5, получили в большинстве документов IETF статус "нерекомендуемые" в связи с некоторыми теоретическими уязвимостями, опубликованными в начале 2005 года. Тем не менее, наличие этих недостатков не говорит о том, что использовать данные алгоритмы нельзя, и, по факту, устаревшее программное обеспечение продолжает их использовать.

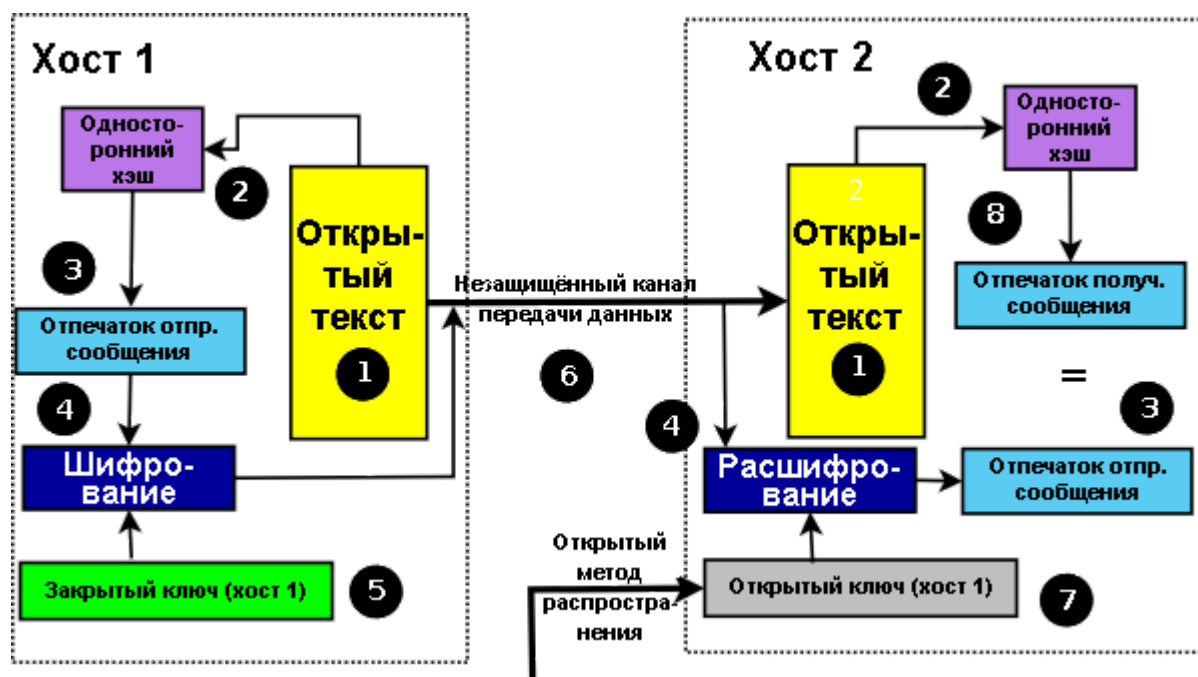


Рисунок 6 — Цифровая подпись

Примечание: Как показано на рисунке, с помощью находящегося в свободном доступе открытого ключа любой может расшифровать сообщение и сделать с него контрольный отпечаток, а зашифровать его может только обладатель закрытого ключа — таким образом обеспечивается гарантия подлинности источника. Лежащий в основе цифровой подписи отпечаток сообщения обеспечивает его целостность, поскольку он зашифрован закрытым ключом отправителя и при передаче изменить его невозможно. Такое применение отличается от обеспечения конфиденциальности сообщения, при котором отправитель шифрует данные с помощью открытого ключа получателя, а расшифровать сообщение можно только с помощью закрытого ключа получателя.

Проблемы, комментарии, предположения, исправления (включая битые ссылки) или есть что добавить? Пожалуйста, выкроите время в потоке занятой жизни, чтобы [написать нам](#), [вебмастеру](#) или в [службу поддержки](#). Оставшийся день Вы проведёте с чувством удовлетворения.

Нашли ошибку в переводе? [Сообщите переводчикам!](#)

